

Milestone 3

Deploying and Monitoring AI Applications

1. Project Overview	2
What the project is	3
Purpose and objectives	3
Technologies/languages used	3
GitHub	3
2. Project Structure	4
Explanation of what's inside each folder/file	4
How the files relate to each other	4
3. Development Process	5
How the application was created	5
Major features implemented	5
Libraries/frameworks used	5
Important design or technical decisions	5
4. How to Install and Run	6
How the model was trained / saved:	6
Prerequisites	6
Installation commands	6
5. How the Application Works	7
Explanation of the main functionality	7
User workflow	7
6. Testing	8
What is tested	8
Test cases/results	8
6. Conclusion	9
What was achieved	9
Challenges encountered	9
Potential future improvements	9

1. Project Overview

What the project is

This project is an end-to-end Machine Learning web application that accepts user-uploaded images and classifies them into one of 10 standard categories using a pre-trained Keras Deep Learning model.

Purpose and objectives

- **Objective:** Deploy a trained Convolutional Neural Network (CNN) image classification model via a lightweight web interface.
- **Purpose:** Provide an easy-to-use interface for real-time image inference without requiring users to run Python scripts or Machine Learning code manually.

Technologies/languages used

- **Language:** Python 3.12
- **Frontend:** HTML5, CSS3, JavaScript (Fetch API)
- **Backend Framework:** FastAPI, Starlette
- **ASGI Server:** Uvicorn
- **Machine Learning & Image Processing:** TensorFlow/Keras, NumPy, Pillow (PIL), Scikit-Learn
- **Templating Engine:** Jinja2

GitHub

<https://github.com/ikeyareinaldosae/ITC501---Artificial-Intelligence-Applications>

2. Project Structure

```
/content/
├── drive/
│   └── MyDrive/
│       └── models/
│           └── cifar10_model.keras      # Saved TensorFlow/Keras trained model
├── app/
│   └── main.py                        # Main FastAPI backend application logic
└── templates/
    └── index.html                     # HTML frontend form and preview script
```

Explanation of what's inside each folder/file

- **drive/MyDrive/models/cifar10_model.keras**: The pre-trained Keras model stored in Google Drive containing the network architecture and learned weights.
- **app/main.py**: The central application script containing route definitions, model loading, and image preprocessing logic.
- **templates/index.html**: The user interface rendered by Jinja2, handling user image selection, dynamic image preview, and asynchronous POST requests to the backend.

How the files relate to each other

When a user accesses the root route (/), **main.py** uses **Jinja2Templates** to render **index.html**. When an image is uploaded through the web page, JavaScript sends a **POST** payload containing the raw image file to the **/predict** route in **main.py**. The backend processes the image, passes it to **cifar10_model.keras**, and sends JSON confidence scores back to **index.html** to update the user UI dynamically.

3. Development Process

How the application was created

1. **Model Storage Integration:** Mounted Google Drive to load the `.keras` model directly into the Colab runtime environment.
2. **Backend Development:** Constructed a FastAPI application with two primary endpoints (`/` for serving HTML and `/predict` for processing inference).
3. **Frontend Integration:** Built an HTML interface with an instant client-side image preview and an asynchronous `fetch()` API call.
4. **Testing and Iteration:** Adjusted payload handling from tabular form fields to binary file upload buffers (`UploadFile` and `PIL.Image`).

Major features implemented

- Real-time client-side image preview before submission.
- Asynchronous file upload handling using `python-multipart`.
- Automatic image scaling to 32 x 32 pixels and RGB conversion to match CIFAR-10 standard inputs.
- Dynamic confidence rating output presented as a percentage.

Libraries/frameworks used

- **FastAPI:** Handles routing and request validation.
- **TensorFlow (Keras):** Loads the neural network and performs tensor inference.
- **Pillow (PIL):** Opens, converts, and resizes raw upload streams.
- **NumPy:** Normalizes pixel values (0-255 -> 0.0-1.0) and manages array shapes.

Important design or technical decisions

- **Transition from Form Data to File Streams:** Replaced tabular numeric inputs with image byte handling (`UploadFile = File(...)`) to support Computer Vision models.
- **Asynchronous Web Processing:** Utilized `async/await` syntax for reading uploaded image bytes to ensure non-blocking backend execution.

4. How to Install and Run

How the model was trained / saved:

The model was trained in `11.4 Deploying and Monitoring AI Applications.ipynb` that can be located and referenced from my Github repository.

```
# Mounting google drive to save the model somewhere for reusability
from google.colab import drive
drive.mount('/content/drive')

... Mounted at /content/drive

model_path = '/content/drive/MyDrive/models/cifar10_model.keras'

model.save(model_path)
```

Prerequisites

- Google Colab environment or local Python 3.10+ runtime.
- Mounted Google Drive containing your `.keras` model file. For this stage the model file is included in the folder or in the github.

Installation commands

Execute the following installation in your terminal or Colab cell:

```
pip install fastapi uvicorn[standard] tensorflow jinja2 python-multipart
pillow numpy
```

5. How the Application Works

Explanation of the main functionality

1. **Input:** The user uploads a visual image file via the HTML web interface.
2. **Preprocessing:** The backend receives the byte stream via `UploadFile`, opens it using `PIL`, converts it to RGB space, resizes it to 32 x 32, scales float values to [0.0, 1.0], and adds a batch dimension (1, 32, 32, 3).
3. **Inference:** `model.predict()` evaluates the array against the 10 CIFAR-10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).
4. **Response:** `np.argmax()` locates the highest prediction index and returns the corresponding text label along with the confidence percentage.

User workflow

```
[Select Image File] → [Client Preview] → [Click "Submit"] → [API POST /predict] → [TensorFlow Inference] → [JSON Result Displayed]
```

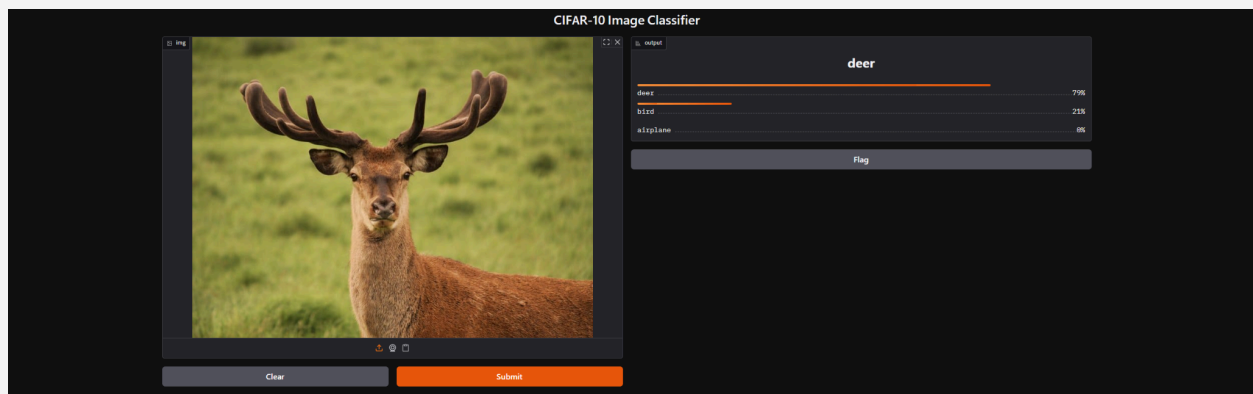


6. Testing

What is tested

- **File Input Types:** Validated system behavior with various formats (.png, .jpeg, .webp).
- **Resolution Variations:** Verified automatic resizing from high-resolution photographs down to required 32 x 32 tensor inputs.
- **Server Routing:** Ensured the ASGI application correctly imports and runs without path errors.

Test cases/results



6. Conclusion

What was achieved

- Successfully built and served a web application for image classification using Gradio and TensorFlow.
- Created a clean front-end user experience with built-in drag-and-drop functionality and probability displays.

Challenges encountered

- **File Directory Mismatches:** Resolved initial path execution issues by verifying script paths within the Google Colab file system.
- **Data Formatting:** Adapted data outputs to return a Python dictionary compatible with Gradio's `Label` component visualization.

Potential future improvements

- Implement sample image presets within the Gradio interface for quick demo testing.
- Train a higher-resolution base model (e.g., ResNet50 or MobileNetV2) to better retain fine details from user uploads.